GRADO EN INTELIGENCIA ARTIFICIAL · UNIR

Inteligencia Artificial e Ingeniería del Conocimiento

Actividad 1

Programación lógica en Python

Diagnóstico médico con lógica de primer orden y aima-python

Estudiante Hugo Gómez Bande

Tipo Laboratorio

Fecha de entrega 23 de noviembre de 2026

Índice

Introducción ... 3

1. Instalación de aima-python ... 3

1.1. Clonar el repositorio ... 3

1.2. Crear un entorno virtual e instalar las dependencias ... 3

1.3. Descargar los datos (submódulo aima-data) ... 4

1.4. Comprobar la instalación con los tests ... 4

1.5. Instalar aima como paquete ... 4

2. Base de conocimiento: gripe y resfriado ... 5

2.1. Predicados ... 5

2.2. Código ... 5

2.3. Consultas y resultados ... 6

3. Base de conocimiento de una enfermedad elegida: el Parkinson ... 7

3.1. Por qué el Parkinson ... 7

3.2. Predicados ... 7

3.3. Código ... 7

3.4. Resultados ... 8

4. Análisis ético del diagnóstico médico automatizado ... 8

5. Uso de herramientas de inteligencia artificial ... 9

Referencias ... 9

Introducción

En esta actividad se construye un pequeño sistema de diagnóstico médico basado en lógica con la biblioteca aima-python, que acompaña al libro *Artificial Intelligence: A Modern Approach* (Russell y Norvig). Se representan síntomas, enfermedades y la relación de causa entre ellos como predicados de lógica de primer orden, se guardan como hechos en una base de conocimiento y se consulta la base para obtener, por ejemplo, todos los síntomas que produce una enfermedad. Es la idea de los sistemas expertos: el conocimiento se escribe de forma explícita y el sistema razona sobre él, de modo que cada respuesta se puede justificar con los hechos que la sostienen.

1 Instalación de aima-python

Equipo: Ubuntu 26.04 con Python 3.14.4 y git. La instalación se hizo desde la terminal el 10 y el 11 de noviembre de 2026.

1.1. Clonar el repositorio

Se crea una carpeta para el proyecto y se descarga dentro el repositorio oficial de aima-python (aimacode/aima-python en GitHub). `git clone` descarga una copia completa del repositorio; el punto final indica que se haga en la carpeta actual, sin crear otra dentro. Los primeros intentos fallaron porque escribí solo el nombre del repositorio (`git clone aima-python`), y git necesita la dirección completa.

```
TERMINAL
cd ~/Downloads
mkdir Aima-python
cd Aima-python
git clone https://github.com/aimacode/aima-python.git .

TERMINAL · COMPROBACIÓN DEL 11/11/2026
$ git remote -v
origin  https://github.com/aimacode/aima-python.git (fetch)
origin  https://github.com/aimacode/aima-python.git (push)
$ git log -1 --format="%h %ad %s" --date=short
49ebbc3 2026-11-01 Run the test matrix on ubuntu-24.04
```

Figura 1. *El repositorio clonado apunta al oficial de aimacode; último commit descargado.*

1.2. Crear un entorno virtual e instalar las dependencias

El primer intento de instalar las dependencias directamente con `pip install -r requirements.txt` no funcionó. Ubuntu no trae pip instalado y, además, protege el Python del sistema para que no se le instalen paquetes, porque lo usan sus propias herramientas y podrían romperse (norma PEP 668, «entorno gestionado externamente»). La solución es crear un **entorno virtual**: un Python aislado solo para este proyecto, en el que lo que se instala no afecta al sistema ni a otros proyectos. Para crearlo hace falta el paquete `python3.14-venv`.

```
TERMINAL
sudo apt update
sudo apt install python3.14-venv
python3 -m venv venv              # crea el entorno en la carpeta venv
source venv/bin/activate          # entra en el entorno
pip install -r requirements.txt   # instala las librerías que usa aima
```

`requirements.txt` es la lista de librerías de las que depende el proyecto (numpy, networkx, tensorflow, pytest…); pip las descarga e instala todas dentro del entorno.

```
TERMINAL · COMPROBACIÓN DEL 11/11/2026
$ pip list | grep -E "numpy|networkx|tensorflow|keras|pytest|matplotlib"
keras                     3.15.1
matplotlib                3.11.2
networkx                  3.7
numpy                     2.5.4
pytest                    9.2.0
tensorflow                2.22.0rc0
$ pip check
No broken requirements found.
```

Figura 2. *Algunas de las dependencias instaladas en el entorno virtual; pip check confirma que no falta ninguna.*

1.3. Descargar los datos (submódulo aima-data)

Los datos que usan los ejemplos y los tests están en otro repositorio, aima-data, enganchado al principal como **submódulo** de git. Al clonar no se descarga, así que hay que pedirlo aparte:

```
TERMINAL
git submodule init
git submodule update

TERMINAL · COMPROBACIÓN DEL 11/11/2026
$ git submodule status
 f4bbabd1962c5022fa8c803abe9aa7edce1ea2a9 aima-data (heads/master)
```

Figura 3. *El submódulo aima-data descargado.*

1.4. Comprobar la instalación con los tests

El proyecto incluye tests automáticos. Se instala pytest y se ejecutan: si pasan, la instalación funciona. Para el módulo de lógica, que es el que usa esta actividad, pasan los 45 tests.

```
TERMINAL
pip install pytest
py.test
python -m pytest tests/test_logic.py      # 45 passed

TERMINAL · COMPROBACIÓN DEL 11/11/2026
$ python -m pytest tests/test_logic.py
platform linux -- Python 3.14.4, pytest-9.2.0, pluggy-1.7.0
collected 45 items

tests/test_logic.py .............................................  [100%]

============================== 45 passed in 2.23s ==============================
```

Figura 4. *Los 45 tests del módulo de lógica pasan.*

1.5. Instalar aima como paquete

Para poder escribir `from aima.logic import ...` desde la carpeta de la actividad, y no solo desde la del proyecto, se instala el propio aima dentro del entorno virtual. La opción `-e` (*editable*) hace que el paquete apunte a la carpeta del repositorio en lugar de copiarla.

```
TERMINAL
pip install -e .
mkdir -p ~/actividad1
```

```
TERMINAL · COMPROBACIÓN DEL 11/11/2026
$ pip show aima
Name: aima
Version: 4.0.0
Editable project location: /home/hugo/Downloads/Aima-python
```

Figura 5. *aima instalado en modo editable dentro del entorno virtual.*

2 Base de conocimiento: gripe y resfriado

2.1. Predicados

Un **predicado** expresa una propiedad de un objeto o una relación entre objetos, y puede ser verdadero o falso. He usado tres:

Predicado	Arg.	Significado	Tipo
`Sintoma(x)`	1	x es un síntoma	Propiedad: clasifica un objeto
`Enfermedad(x)`	1	x es una enfermedad	Propiedad: clasifica un objeto
`Causa(e, s)`	2	la enfermedad e causa el síntoma s	Relación entre dos objetos

Cada predicado tiene una sola función: los de un argumento dicen *qué es* cada cosa y `Causa` dice *cómo se relacionan*. Por eso `Sintoma` lleva un único argumento: si incluyera también la enfermedad, repetiría lo que ya expresa `Causa`. El orden de los argumentos de `Causa` se fija una vez (primero la enfermedad, después el síntoma) y se respeta siempre, porque `Causa(Gripe, Tos)` no significa lo mismo que `Causa(Tos, Gripe)`.

En aima, los términos que empiezan por mayúscula son **constantes**, objetos concretos (`Gripe`, `Fiebre`), y los que empiezan por minúscula son **variables** (`x`), huecos que el sistema intenta rellenar al consultar. Las constantes no pueden llevar espacios, por eso «dolor de garganta» se escribe `DolorDeGarganta`.

Aunque el enunciado habla de lógica proposicional, esta representación es de **lógica de primer orden**: la proposicional solo maneja afirmaciones completas, verdaderas o falsas («la gripe causa fiebre»), mientras que la de primer orden distingue objetos, predicados y variables. Eso es lo que permite hacer preguntas generales como «¿qué síntomas causa el resfriado?».

2.2. Código

```python
# Actividad 1 · Programación lógica en Python (aima-python)
# Diagnóstico médico con lógica de primer orden
# Autor: Hugo Gómez Bande

# ── Importaciones ──────────────────────────────────────────────
# FolKB: base de conocimiento de lógica de primer orden (First-Order Logic Knowledge Base)
# fol_bc_ask: consulta la base por encadenamiento hacia atrás (backward chaining)
# expr: convierte un texto en una expresión lógica que aima entiende
from aima.logic import FolKB, fol_bc_ask
from aima.utils import expr


# == PARTE 1 · Gripe y resfriado ================================
```

```
# Base de conocimiento: empieza vacía y se le añaden hechos con tell()
kb = FolKB()

# Predicado Sintoma(x): «x es un síntoma».
# Un solo argumento: clasifica un objeto. Las constantes van en mayúscula
# y sin espacios (DolorDeGarganta).
kb.tell(expr("Sintoma(Fiebre)"))
kb.tell(expr("Sintoma(Tos)"))
kb.tell(expr("Sintoma(DolorDeGarganta)"))

# Predicado Enfermedad(x): «x es una enfermedad».
kb.tell(expr("Enfermedad(Gripe)"))
kb.tell(expr("Enfermedad(Resfriado)"))

# Predicado Causa(e, s): «la enfermedad e causa el síntoma s».
# Dos argumentos: es una relación, siempre en el orden (enfermedad, síntoma).
kb.tell(expr("Causa(Gripe, Fiebre)"))
kb.tell(expr("Causa(Gripe, Tos)"))
kb.tell(expr("Causa(Gripe, DolorDeGarganta)"))
kb.tell(expr("Causa(Resfriado, Tos)"))
kb.tell(expr("Causa(Resfriado, DolorDeGarganta)"))

# Comprobación de la base: todos los síntomas y todas las enfermedades.
# x (en minúscula) es una variable: fol_bc_ask busca los valores que la
# hacen verdadera y devuelve un generador de sustituciones {x: valor};
# list() las recoge todas para poder imprimirlas.
print(list(fol_bc_ask(kb, expr("Sintoma(x)"))))
print(list(fol_bc_ask(kb, expr("Enfermedad(x)"))))

# Consulta pedida: todos los síntomas que produce el resfriado.
# La enfermedad es fija (Resfriado) y el síntoma es la variable.
print(list(fol_bc_ask(kb, expr("Causa(Resfriado, x)"))))
```

2.3. Consultas y resultados

`kb.tell` añade a la base **hechos**, afirmaciones que se dan por ciertas sin condiciones. Para consultar se usa `fol_bc_ask`, que aplica **encadenamiento hacia atrás** (*backward chaining*): parte de la pregunta y busca en la base los hechos (o reglas) que la demuestran.

La respuesta no es verdadero o falso, sino una lista de **sustituciones**: qué valor debe tomar cada variable para que la pregunta sea cierta. `fol_bc_ask` las genera de una en una (devuelve un generador), por lo que se usa `list()` para obtenerlas todas; sin `list()`, `print` solo muestra `<generator object ...>`. Así se leen los tres tipos de respuesta:

Consulta	Respuesta	Significado
`Causa(Gripe, Fiebre)`	`[{}]`	Sí: no hay variables que sustituir.
`Causa(Resfriado, x)`	`[{x: Tos}, {x: DolorDeGarganta}]`	Sí, con dos valores posibles de x.
Un hecho que no está en la base, p. ej. `Causa(Gripe, Nauseas)`	`[]`	No se puede demostrar.

Resultado del programa: las dos comprobaciones de la base (todos los síntomas y todas las enfermedades) y la consulta pedida, los síntomas del resfriado:

SALIDA DEL PROGRAMA
```
[{x: Fiebre}, {x: Tos}, {x: DolorDeGarganta}]
[{x: Gripe}, {x: Resfriado}]
[{x: Tos}, {x: DolorDeGarganta}]
```

El resfriado produce tos y dolor de garganta, que coincide con los hechos introducidos.

3 Base de conocimiento de una enfermedad elegida: el Parkinson

3.1. Por qué el Parkinson

He elegido el Parkinson porque es muy distinto de la gripe y el resfriado: no es una infección, sino una enfermedad crónica y neurodegenerativa, y sus síntomas principales son motores (temblor, rigidez y lentitud de movimientos). Además, no existe un análisis que la confirme por sí solo: el diagnóstico es sobre todo clínico, a partir del examen neurológico. Por eso tiene sentido ampliar la base con información que no aparecía en la parte anterior: factores de riesgo y pruebas.

3.2. Predicados

Se reutilizan `Enfermedad`, `Sintoma` y `Causa` con el mismo significado que en la parte 1, para que el diseño sea coherente, y se añaden dos predicados nuevos:

Predicado	Significado	Hechos
`FactorDeRiesgo(e, f)`	f aumenta la probabilidad de padecer e, pero no la causa	Edad avanzada, antecedentes familiares
`Prueba(e, p)`	p es una prueba que se usa para estudiar e	Examen neurológico, resonancia magnética

Los factores de riesgo no se expresan con `Causa` a propósito: la edad avanzada no provoca el Parkinson, solo lo hace más probable. Confundir ambas cosas sería un error de representación del conocimiento. Los hechos se guardan en una base de conocimiento propia (`kb2`), separada de la de la gripe y el resfriado.

3.3. Código

PYTHON
```python
# == PARTE 2 · Enfermedad elegida: Parkinson ==========================

# Base de conocimiento independiente para la segunda enfermedad
kb2 = FolKB()

# Mismos predicados y con el mismo significado que en la parte 1
kb2.tell(expr("Enfermedad(Parkinson)"))
kb2.tell(expr("Sintoma(Temblor)"))
kb2.tell(expr("Sintoma(Rigidez)"))
kb2.tell(expr("Sintoma(LentitudDeMovimientos)"))
kb2.tell(expr("Causa(Parkinson, Temblor)"))
kb2.tell(expr("Causa(Parkinson, Rigidez)"))
kb2.tell(expr("Causa(Parkinson, LentitudDeMovimientos)"))

# Predicado nuevo FactorDeRiesgo(e, f): f aumenta la probabilidad de
```

```
# padecer e, pero no la causa (por eso no se expresa con Causa).
kb2.tell(expr("FactorDeRiesgo(Parkinson, EdadAvanzada)"))
kb2.tell(expr("FactorDeRiesgo(Parkinson, AntecedentesFamiliares)"))

# Predicado nuevo Prueba(e, p): prueba p que se usa para estudiar e.
kb2.tell(expr("Prueba(Parkinson, ExamenNeurologico)"))
kb2.tell(expr("Prueba(Parkinson, ResonanciaMagnetica)"))

# Consultas: síntomas, factores de riesgo y pruebas del Parkinson
print(list(fol_bc_ask(kb2, expr("Causa(Parkinson, x)"))))
print(list(fol_bc_ask(kb2, expr("FactorDeRiesgo(Parkinson, x)"))))
print(list(fol_bc_ask(kb2, expr("Prueba(Parkinson, x)"))))
```

3.4. Resultados

SALIDA DEL PROGRAMA

```
[{x: Temblor}, {x: Rigidez}, {x: LentitudDeMovimientos}]
[{x: EdadAvanzada}, {x: AntecedentesFamiliares}]
[{x: ExamenNeurologico}, {x: ResonanciaMagnetica}]
```

Las tres consultas devuelven los síntomas, los factores de riesgo y las pruebas introducidos, lo que comprueba que los predicados nuevos funcionan igual que los de la parte 1.

4 Análisis ético del diagnóstico médico automatizado

Mi postura es que un sistema de diagnóstico es mucho más fiable cuando se construye a partir del conocimiento de las personas que saben de medicina y se consigue trasladar ese conocimiento correctamente a la máquina. Es justo lo que hace un sistema basado en lógica como el de esta actividad: los hechos y las relaciones (qué enfermedad causa qué síntoma) los aportan personas, y la máquina se limita a razonar con ellos de forma explícita. Por eso cada respuesta se puede justificar: si el sistema dice que el resfriado produce tos, es porque esa relación está escrita en la base de conocimiento y cualquiera puede comprobarla.

Creo que una gran parte del diagnóstico y del triaje debería ser un proceso refinado, mecánico y calculado, que haya demostrado funcionar. Un proceso así aplica siempre los mismos criterios, sin depender del cansancio, de la prisa o de la experiencia de quien esté de guardia ese día. En el triaje, por ejemplo, decidir la prioridad de cada paciente con criterios fijos y medidos me parece más justo y más seguro que dejarlo a la impresión de cada momento.

Eso no significa eliminar el factor humano. Hay casos dudosos en los que los datos no bastan para decidir, y ahí el instinto y la experiencia clínica del profesional pueden servir para decantarse finalmente. Mi propio sistema lo muestra: la consulta `Causa(x, Tos)` devuelve a la vez la gripe y el resfriado, sin indicar cuál es más probable. Ante un empate así, la decisión final debe tomarla una persona.

Por último, para que la medicina pueda avanzar y revisar sus métodos, necesita una organización replicable que pueda revisarse con criterios estandarizados. Un sistema basado en reglas explícitas cumple esto mejor que las decisiones individuales: con los mismos datos da siempre el mismo resultado, y si se equivoca se puede localizar qué hecho o qué regla era incorrecto y corregirlo para todos los casos futuros. Así los errores se convierten en mejoras del método, en lugar de quedarse en la experiencia de una sola persona.

En resumen, creo que un sistema de diagnóstico automatizado debería cumplir estos principios: basarse en conocimiento experto bien trasladado, demostrar que funciona antes de usarse, ser transparente para

poder explicar cada conclusión, poder revisarse con criterios estandarizados y dejar la última palabra a una persona en los casos dudosos. La máquina aporta la base sistemática y el profesional, el juicio en lo que no se puede reducir a reglas.

5 Uso de herramientas de inteligencia artificial

Para esta actividad he utilizado un asistente de inteligencia artificial (Claude, de Anthropic) como tutor: me explicó los conceptos de la biblioteca (bases de conocimiento, consultas, sustituciones), me corrigió el código mientras lo escribía, redactó a partir de lo trabajado en esas sesiones las explicaciones de los apartados 1 a 3, añadió los comentarios del código y las consultas de factores de riesgo y pruebas del Parkinson, y redactó el apartado 4 a partir de mi opinión. El diseño de los predicados, los hechos de las dos bases de conocimiento, las consultas de los síntomas, la elección de la enfermedad y las ideas del análisis ético son míos, y he revisado todo el texto. El trabajo completo, paso a paso, está en el historial del repositorio git de la actividad: https://github.com/Hugelidus/ia-actividad1-logica (privado; acceso a petición).

Referencias

Russell, S. y Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson.

aimacode. *aima-python: Python implementation of algorithms from Russell and Norvig's Artificial Intelligence: A Modern Approach*. Repositorio de GitHub. https://github.com/aimacode/aima-python (versión usada: commit 49ebbc3, 1 de noviembre de 2026).

aimacode. *aima-data: Data files for the AIMA code repositories*. Repositorio de GitHub. https://github.com/aimacode/aima-data